Financial Risk Analysis with Python Goldman Sachs

TASK 1: Data Claening And Formating

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import ttest_ind, zscore
from datetime import datetime
```

In this step imported required libraries

```
df = pd.read_csv("goldman_sachs.csv")
df.head()
```

```
   TransactionID CustomerID AccountID AccountType TransactionType
0             33   CUST6549  ACC12334      Credit      Withdrawal
1            177   CUST2942  ACC52650      Credit      Withdrawal
2            178   CUST6776  ACC45101     Current         Deposit
3            173   CUST2539  ACC88252     Current      Withdrawal
4             67   CUST2626  ACC21878     Savings      Withdrawal

           Product    Firm   Region    Manager TransactionDate
0  Savings Account  Firm C  Central  Manager 1      21-10-2023
1        Home Loan  Firm A     East  Manager 3      20-06-2023
2    Personal Loan  Firm C    South  Manager 3      02-01-2023
3      Mutual Fund  Firm A  Central  Manager 2      25-07-2023
4        Home Loan  Firm C  Central  Manager 4      25-07-2023

   TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths
0        87480.05448     74008.43310   0.729101           319           200
1        20315.74505     22715.83590   0.472424           692            47
2        10484.57165     42706.09210   0.648784           543           109
3        45122.27373    114176.56870   0.734832           430           103
4        42360.79878     17863.02644   0.289304           468           234
```

Here data is imported

1.1 and 1.2 Remove/treat any special characters or non-numeric entries from financial fields

```
cols = ['TransactionAmount', 'AccountBalance', 'RiskScore']


for col in cols:
    df[col] = (
        df[col]
        .astype(str)
        .str.replace(r'[^0-9.-]', '', regex=True)
        .astype(float)
    )

df.to_csv("Cleaned_Data.csv", index=False)

cleaned_df =pd.read_csv("Cleaned_Data.csv")
```

```
   TransactionID CustomerID AccountID AccountType TransactionType
0             33   CUST6549  ACC12334      Credit      Withdrawal
1            177   CUST2942  ACC52650      Credit      Withdrawal
2            178   CUST6776  ACC45101     Current         Deposit
3            173   CUST2539  ACC88252     Current      Withdrawal
4             67   CUST2626  ACC21878     Savings      Withdrawal

           Product    Firm   Region    Manager TransactionDate
0  Savings Account  Firm C  Central  Manager 1      21-10-2023
1        Home Loan  Firm A     East  Manager 3      20-06-2023
2    Personal Loan  Firm C    South  Manager 3      02-01-2023
3      Mutual Fund  Firm A  Central  Manager 2      25-07-2023
4        Home Loan  Firm C  Central  Manager 4      25-07-2023

   TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths
0        87480.05448     74008.43310   0.729101           319           200
1        20315.74505     22715.83590   0.472424           692            47
2        10484.57165     42706.09210   0.648784           543           109
3        45122.27373    114176.56870   0.734832           430           103
4        42360.79878     17863.02644   0.289304           468           234
```

1.3 Validate and format date columns.

```
changed_date = cleaned_df['TransactionDate'] =
pd.to_datetime(cleaned_df['TransactionDate'], dayfirst = True,
errors='coerce')

changed_date.to_csv("Task 1_3.csv", index = False)
```

1.4 Ensure account types and transaction categories are standardized.

```
cleaned_df['AccountType'] = cleaned_df['AccountType'].str.lower().str.strip()
cleaned_df['TransactionType'] =
cleaned_df['TransactionType'].str.lower().str.strip()
categories_standardized = cleaned_df.to_csv("Task 1.4.csv")

cleaned_df.head(10)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType
468             73   CUST9209  ACC10117     savings        transfer
25              34   CUST3109  ACC10117     savings         deposit
87             106   CUST8155  ACC10117      credit         deposit
742            169   CUST3810  ACC10117        loan         deposit
384            158   CUST3041  ACC10996     savings      withdrawal
294             93   CUST6776  ACC10996     savings         payment
396             88   CUST6776  ACC10996      credit        transfer
574             22   CUST6565  ACC10996     current      withdrawal
59              26   CUST9843  ACC10996      credit         deposit
557            115   CUST8091  ACC11062      credit         payment

              Product    Firm   Region    Manager TransactionDate  ...
468       Credit Card  Firm E     West  Manager 1      2023-01-30  ...
25          Home Loan  Firm E     West  Manager 3      2023-02-21  ...
87        Mutual Fund  Firm C     East  Manager 3      2023-06-18  ...
742         Home Loan  Firm B     East  Manager 4      2024-03-14  ...
384       Credit Card  Firm B    South  Manager 3      2023-05-08  ...
294   Savings Account  Firm C    North  Manager 4      2023-10-19  ...
396         Home Loan  Firm C    North  Manager 2      2023-10-21  ...
574       Mutual Fund  Firm A  Central  Manager 2      2024-01-29  ...
59        Credit Card  Firm E    North  Manager 4      2024-02-09  ...
557       Mutual Fund  Firm B  Central  Manager 1      2023-11-08  ...

     TenureMonths    Month  Year  Txn_Category    Gap Inactive_Flag  \
468            60  2023-01  2023         debit    NaN    Active acc
25             85  2023-02  2023        credit   22.0    Active acc
87            109  2023-06  2023        credit  117.0  Inactive acc
742           218  2024-03  2024        credit  270.0  Inactive acc
384           233  2023-05  2023         debit    NaN    Active acc
294           103  2023-10  2023         debit  164.0  Inactive acc
396           207  2023-10  2023         debit    2.0    Active acc
574           201  2024-01  2024         debit  100.0  Inactive acc
59            112  2024-02  2024        credit   11.0    Active acc
557            71  2023-11  2023         debit    NaN    Active acc

     CreditAmount  DebitAmount  TxnNature   Z_Score
```


```
468       0.00000  57310.76365      Debit  0.197382
25    67555.28072      0.00000     Credit  0.549962
87    63248.56067      0.00000     Credit  0.401740
742   11366.36239      0.00000     Credit -1.383863
384       0.00000  11989.50230      Debit -1.362417
294       0.00000  61653.06253      Debit  0.346828
396       0.00000  61545.77749      Debit  0.343136
574       0.00000  52970.34507      Debit  0.048000
59    62580.86356      0.00000     Credit  0.378760
557       0.00000  15601.70053      Debit -1.238098

[10 rows x 24 columns]
```

we clean the data by fixing numbers, dates, and categories so the dataset is accurate and ready to use for analysis.

Task 2: Descriptive Transactional Analysis

2.1 Calculate monthly and yearly summaries of total credits, debits, and net transaction

```
cleaned_df['Month'] = cleaned_df['TransactionDate'].dt.to_period('M')
cleaned_df['Year'] = cleaned_df['TransactionDate'].dt.year

summary = cleaned_df.pivot_table(
    values='TransactionAmount',
    index='Month',
    columns='TransactionType',
    aggfunc='sum'
)
summary['Net'] = summary.get('credit',0) - summary.get('debit',0)

summary.round(2)
```

```
TransactionType    deposit    payment   transfer  withdrawal  Net
Month
2023-01          738270.10  767715.19  503614.96  1047046.34    0
2023-02          648261.00  354836.66  286550.14   792466.61    0
2023-03          604002.42  865951.65  545314.71   616527.07    0
2023-04          439321.69  590121.52  301770.39   446402.37    0
2023-05          425589.87  906411.88  731136.02   566167.37    0
2023-06          469388.81  214746.05  175645.94   485198.42    0
2023-07          648027.88  175522.17  416423.61   315287.01    0
2023-08          544970.36  607459.32  719074.09   684698.13    0
2023-09          712838.80  411925.53  281986.25   918769.32    0
2023-10          689706.22  976609.47  659411.80   830495.02    0
```

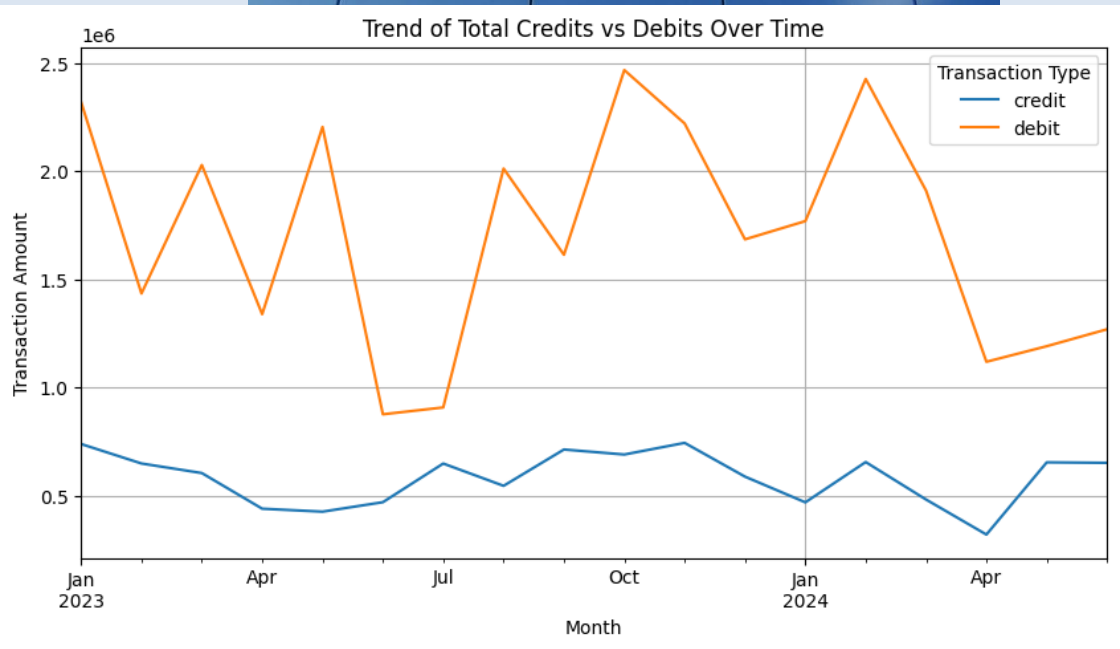

```
2023-11          743507.57  648791.19  689144.30   881503.51    0
2023-12          587483.87  727014.86  527649.37   429426.94    0
2024-01          468686.37  860246.22  276849.49   631275.14    0
2024-02          654854.53  854802.73  893580.40   677024.32    0
2024-03          481619.62  988407.38  450494.51   470592.27    0
2024-04          319754.66  309329.22  591125.23   217879.65    0
2024-05          653559.12  171182.43  665752.98   353384.97    0
2024-06          651292.95  151201.43  639957.70   477500.29    0
```

This analysis helps identify transaction trends, high-value customers, and risky accounts to support better financial monitoring and decision-making.

```
summary.to_csv("Task 2_1.csv")
```

2.2 Plot trends in total credits vs. debits over time

```
cleaned_df['Txn_Category'] =cleaned_df['TransactionType'].map({
    'deposit': 'credit',
    'transfer': 'debit',
    'payment': 'debit',
    'withdrawal': 'debit'
})

monthly_summary = cleaned_df.pivot_table(
    values='TransactionAmount',
    index='Month',
    columns='Txn_Category',
    aggfunc='sum'
)

monthly_summary[['credit', 'debit']].plot(figsize=(10,5))

plt.title("Trend of Total Credits vs Debits Over Time")
plt.xlabel("Month")
plt.ylabel("Transaction Amount")
plt.legend(title="Transaction Type")
plt.grid(True)
plt.show()
```



This code classifies transactions into credits and debits, calculates monthly totals for each, and plots their trends over time to compare money inflows and outflows.

2.3 dentify top and bottom performing accounts based on net inflow.

```
account_flow = cleaned_df.groupby('AccountID')['TransactionAmount'].sum()

top_accounts = account_flow.sort_values(ascending=False).head(10)
bottom_accounts = account_flow.sort_values().head(10)

print(top_accounts)
```

```
AccountID
ACC46655    728037.402705
ACC33287    591591.095890
ACC16241    539612.142850
ACC53466    494957.158800
ACC29396    482720.015550
ACC83269    468734.595390
ACC57700    465047.926414
ACC60432    462957.849880
ACC29356    435573.496188
ACC13357    432527.808260
Name: TransactionAmount, dtype: float64
```

```
print(bottom_accounts)
```


```
AccountID
ACC46953   -24811.428950
ACC11062     4014.264900
ACC43771    14033.502560
ACC87602    14944.062400
ACC26956    15603.452120
ACC21429    20442.761880
ACC78089    26828.979199
ACC62809    34979.062450
ACC29007    35207.179489
ACC29646    36055.574280
Name: TransactionAmount, dtype: float64
```

this show the best and worst performing accounts based on net inflow, helping the bank recognize high-value customers and flag accounts with consistently negative cash flow for closer monitoring.

2.4 Identify and flag accounts as dormant or inactive if there is a gap of two months or more between consecutive transactions.

```
cleaned_df = cleaned_df.sort_values(['AccountID','TransactionDate'])

cleaned_df['Gap'] =
cleaned_df.groupby('AccountID')['TransactionDate'].diff().dt.days

cleaned_df['Inactive_Flag'] = cleaned_df['Gap'].apply(lambda x: "Inactive
acc" if x >= 60 else "Active acc")

cleaned_df[["Inactive_Flag","AccountID"]]
```

```
    Inactive_Flag AccountID
468    Active acc  ACC10117
25     Active acc  ACC10117
87   Inactive acc  ACC10117
742  Inactive acc  ACC10117
384    Active acc  ACC10996
..             ...       ...
542  Inactive acc  ACC99409
223    Active acc  ACC99549
504  Inactive acc  ACC99549
706  Inactive acc  ACC99549
510  Inactive acc  ACC99549

[800 rows x 2 columns]
```

This analysis identifies dormant or inactive accounts based on long gaps between transactions, helping the bank flag inactive customers for re-engagement or risk monitoring.

```
cleaned_df.to_csv("Task 2.4.csv")
```

3 Customer Profile Building

3.1 Group accounts by activity levels: High, Medium, Low based on transaction frequency on your analysis and rubrics. Do not forget to mention the rubric in the headings.

```
transction_frequency = (
    cleaned_df
    .groupby('AccountID')
    .size()
    .reset_index(name='TxnCount')
)
def activity_level(count):
    if count > 20:
        return 'High'
    elif count >= 10:
        return 'Medium'
    else:
        return 'Low'

transction_frequency['ActivityLevel'] = (
    transction_frequency['TxnCount'].apply(activity_level)
)
activity_balance = transction_frequency.merge(
    cleaned_df.groupby('AccountID')['AccountBalance']
        .mean()
        .reset_index(),
    on='AccountID'
)

transction_frequency['ActivityLevel'] =
transction_frequency['TxnCount'].apply(activity_level)

transction_frequency[["ActivityLevel","AccountID"]]
```

```
    ActivityLevel AccountID
0             Low  ACC10117
1             Low  ACC10996
2             Low  ACC11062
3             Low  ACC11188
4             Low  ACC11285
```


```
..            ...       ...
189           Low  ACC97225
190           Low  ACC97411
191           Low  ACC99117
192           Low  ACC99409
193           Low  ACC99549

[194 rows x 2 columns]
```

This analysis groups accounts into high, medium, and low activity levels based on transaction frequency, helping identify highly engaged customers and low-activity accounts that may need re-engagement or closer monitoring.

```
transction_frequency.to_csv("Task 3_1.csv")
```

3.2 Segment customers by average balance and transaction volume

```
customer_profile = cleaned_df.groupby('AccountID').agg(
    AvgBalance=('AccountBalance', 'mean'),
    TotalTxnVolume=('TransactionAmount', 'sum')
).reset_index()

balance_median = customer_profile['AvgBalance'].median()
volume_median = customer_profile['TotalTxnVolume'].median()

def segment_customer(row):
    if row['AvgBalance'] > balance_median and row['TotalTxnVolume'] >
volume_median:
        return 'High Balance – High Volume'
    elif row['AvgBalance'] > balance_median and row['TotalTxnVolume'] <=
volume_median:
        return 'High Balance – Low Volume'
    elif row['AvgBalance'] <= balance_median and row['TotalTxnVolume'] >
volume_median:
        return 'Low Balance – High Volume'
    else:
        return 'Low Balance – Low Volume'

customer_profile['CustomerSegment'] =
customer_profile.apply(segment_customer, axis=1)

segment_summary = customer_profile['CustomerSegment'].value_counts()
segment_summary
```

```
CustomerSegment
Low Balance – Low Volume      49
High Balance – High Volume    49
Low Balance – High Volume     48
```


```
High Balance – Low Volume     48
Name: count, dtype: int64
```

```
customer_profile
```

	AccountID	AvgBalance	TotalTxnVolume	CustomerSegment
0	ACC10117	70107.007957	199480.967430	Low Balance – Low Volume
1	ACC10996	43568.008084	250739.550950	Low Balance – High Volume
2	ACC11062	38137.132610	4014.264900	Low Balance – Low Volume
3	ACC11188	69652.151044	257576.603590	Low Balance – High Volume
4	ACC11285	97401.348560	96729.609841	High Balance – Low Volume
..	...	...	...	...
189	ACC97225	38652.306677	160282.714280	Low Balance – Low Volume
190	ACC97411	55978.315635	174551.560470	Low Balance – Low Volume
191	ACC99117	47228.185087	212848.743280	Low Balance – High Volume
192	ACC99409	83743.915565	113191.225750	High Balance – Low Volume
193	ACC99549	68641.201433	188942.411270	Low Balance – Low Volume

```
[194 rows x 4 columns]
```

This segmentation divides customers into four groups based on average balance and transaction volume, helping the bank identify high-value customers, frequent low-balance users, and low-engagement accounts for targeted strategies.

```
customer_profile.to_csv("Task 3_2.csv")
```

3.3.1 High-net inflow accounts

```
# Separate credit and debit amounts
cleaned_df['CreditAmount'] = cleaned_df.apply(
    lambda x: x['TransactionAmount'] if x['Txn_Category'] == 'credit' else 0,
    axis=1
)

cleaned_df['DebitAmount'] = cleaned_df.apply(
    lambda x: x['TransactionAmount'] if x['Txn_Category'] == 'debit' else 0,
    axis=1
)

# Net inflow per account
net_inflow = cleaned_df.groupby('AccountID').agg(
    TotalCredit=('CreditAmount', 'sum'),
    TotalDebit=('DebitAmount', 'sum')
)

net_inflow['NetInflow'] = net_inflow['TotalCredit'] -
net_inflow['TotalDebit']
```


```
# High-net inflow accounts
high_net_inflow_accounts = net_inflow[net_inflow['NetInflow'] > 0]

high_net_inflow_accounts
```

AccountID	TotalCredit	TotalDebit	NetInflow
ACC10117	142170.20378	57310.763650	84859.440130
ACC15359	122856.48300	120413.197880	2443.285120
ACC18140	45680.91092	38542.747670	7138.163250
ACC19178	64100.78213	0.000000	64100.782130
ACC21264	79561.48165	58077.649160	21483.832490
ACC21878	112600.88690	42360.798780	70240.088120
ACC24508	36340.30994	0.000000	36340.309940
ACC29477	108199.42190	49541.181310	58658.240590
ACC29646	36055.57428	0.000000	36055.574280
ACC30146	84232.18852	38590.385590	45641.802930
ACC31902	204456.94267	126739.664910	77717.277760
ACC33287	390354.42641	201236.669480	189117.756930
ACC38559	119378.42897	56440.995900	62937.433070
ACC40952	72999.23276	0.000000	72999.232760
ACC41829	159142.37375	136824.239310	22318.134440
ACC45101	103802.06452	99556.653000	4245.411520
ACC45951	98505.08125	19454.382670	79050.698580
ACC46953	0.00000	-24811.428950	24811.428950
ACC48501	346856.33960	0.000000	346856.339600
ACC49774	138953.60860	120197.260980	18756.347620
ACC50817	244837.13480	123447.969110	121389.165690
ACC57516	221384.17912	107219.927320	114164.251800
ACC70460	67804.70544	46450.583010	21354.122430
ACC71426	69256.30947	68504.952821	751.356649
ACC71938	62715.18663	41441.868014	21273.318616
ACC73104	55095.76821	14767.436600	40328.331610
ACC75675	98344.75514	50719.981060	47624.774080
ACC76549	159360.47608	105145.554460	54214.921620
ACC81631	70867.26134	57261.715840	13605.545500
ACC83269	242156.81062	226577.784770	15579.025850
ACC87006	245497.37832	101488.368620	144009.009700
ACC87602	33532.36187	-18588.299470	52120.661340
ACC92360	61299.30000	51692.313850	9606.986150
ACC94203	221569.90933	164180.876070	57389.033260
ACC95774	110414.82355	14746.314350	95668.509200
ACC97225	87320.05768	72962.656600	14357.401080
ACC99117	167040.52263	45808.220650	121232.301980

This analysis identifies high net inflow accounts by comparing total credits and debits, helping the bank recognize financially strong customers with consistent positive cash flow for retention and premium services.

```
high_net_inflow_accounts.to_csv("Task 3_3_1.csv")
```

3.3.2 High-frequency low-balance accounts

```
# Transaction frequency per account
transction_frequency = cleaned_df.groupby('AccountID').size()

# Average balance per account
avg_balance = cleaned_df.groupby('AccountID')['AccountBalance'].mean()

# Combine
hf_lb = pd.concat([transction_frequency, avg_balance], axis=1)
hf_lb.columns = ['TxnCount', 'AvgBalance']

# Thresholds
freq_median = hf_lb['TxnCount'].median()
balance_median = hf_lb['AvgBalance'].median()

# High-frequency, low-balance accounts
high_freq_low_balance_accounts = hf_lb[
    (hf_lb['TxnCount'] > freq_median) &
    (hf_lb['AvgBalance'] < balance_median)
]
```

```
high_freq_low_balance_accounts
```

AccountID	TxnCount	AvgBalance
ACC10996	5	43568.008084
ACC11188	5	69652.151044
ACC13357	6	69179.806513
ACC18177	5	63505.219474
ACC23736	7	60801.533102
ACC24070	5	55694.967801
ACC26973	5	58738.210687
ACC28292	10	51228.003570
ACC30787	5	60525.872356
ACC31539	6	45185.938342
ACC32212	6	66264.686983
ACC33287	8	59331.981186
ACC37688	7	63580.556277
ACC45101	5	70039.890212
ACC48303	6	64403.296167


```
ACC48501          5  64565.232000
ACC49774          7  54898.786583
ACC50817          5  69212.723150
ACC53865          5  67573.516150
ACC57700          8  60537.126840
ACC58667          5  57596.212717
ACC61926          6  53209.096892
ACC71388          5  52366.145184
ACC74631          6  50478.579943
ACC75767          6  66138.619752
ACC77773          5  71762.833922
ACC78178          9  69327.319332
ACC78589          5  59110.034406
ACC80131          6  61459.218423
ACC82926          5  48804.796760
ACC83269          7  48187.873590
ACC88252          8  71244.257108
ACC88516          6  60473.425825
ACC94907          7  50272.481959
```

This analysis identifies accounts with frequent transactions but low average balances, highlighting customers who are highly active yet financially constrained and may need balance management support or closer monitoring.

```
high_freq_low_balance_accounts.to_csv("Task 3_3_2.csv")
```

3.3.3. Accounts with negative or near-zero balances

```
zerobalace_account_or_negative_account = avg_balance[
    avg_balance <= 1000
]

zerobalace_account_or_negative_account
```

```
AccountID
ACC19178   -1541.176812
Name: AccountBalance, dtype: float64
```

This analysis identifies accounts with negative or near-zero balances, helping flag financially stressed customers who may require overdraft monitoring, alerts, or corrective action.

```
zerobalace_account_or_negative_account.to_csv("Task 3_3_3.csv")
```

Task 4: Financial Risk Identification

4.1 Track accounts with frequent large withdrawals or overdrafts.

```
cleaned_df['TxnNature'] = cleaned_df['TransactionType'].map({
    'deposit': 'Credit',
    'withdrawal': 'Debit',
    'payment': 'Debit',
    'transfer': 'Debit'
})

# Filter large withdrawals
large_withdrawals = cleaned_df[
    (cleaned_df['TransactionType'] == 'withdrawal') &
    (cleaned_df['TransactionAmount'] > 50000)
]

# Count per account
large_withdrawal_count =
large_withdrawals.groupby('AccountID').size().reset_index(
    name='LargeWithdrawalCount'
)


# View result
large_withdrawal_count.head()
```

```
  AccountID  LargeWithdrawalCount
0  ACC10996                     1
1  ACC11188                     1
2  ACC12334                     1
3  ACC15228                     2
4  ACC15359                     1
```

```
# Filter overdraft transactions
overdrafts = cleaned_df[cleaned_df['AccountBalance'] < 0]

# Count per account
overdraft_count = overdrafts.groupby('AccountID').size().reset_index(
    name='OverdraftCount'
)

# View result
overdraft_count
```

	AccountID	OverdraftCount
0	ACC16241	1
1	ACC19178	1
2	ACC23736	1
3	ACC26973	1
4	ACC28154	1
5	ACC28292	2
6	ACC29477	1
7	ACC33287	1
8	ACC49774	1
9	ACC58667	1
10	ACC70314	1
11	ACC77533	1
12	ACC83005	1
13	ACC88449	1
14	ACC94242	1

This analysis identifies accounts with frequent large withdrawals, helping flag potentially risky or stressed accounts that require closer monitoring for overdrafts or unusual cash outflows.

```
large_withdrawals.to_csv("Task 4_1.csv")
```

4.2 Calculate balance volatility using standard deviation or coefficient of variation.

```
# standard deviation
balance_stats = cleaned_df.groupby('AccountID')['AccountBalance'].agg(
    MeanBalance='mean',
    StdBalance='std'
).reset_index()

balance_stats['CV'] = (
    balance_stats['StdBalance'] / balance_stats['MeanBalance']
)

balance_stats
```

	AccountID	MeanBalance	StdBalance	CV
0	ACC10117	70107.007957	25886.972758	0.369249
1	ACC10996	43568.008084	9434.002316	0.216535
2	ACC11062	38137.132610	3208.737888	0.084137
3	ACC11188	69652.151044	35494.660810	0.509599
4	ACC11285	97401.348560	55922.732441	0.574147
..	...	...	...	...
189	ACC97225	38652.306677	28069.592780	0.726207
190	ACC97411	55978.315635	7871.678922	0.140620
191	ACC99117	47228.185087	20780.582578	0.440004


```
192  ACC99409  83743.915565  21429.756821  0.255896
193  ACC99549  68641.201433  26251.797058  0.382450

[194 rows x 4 columns]
```

This analysis measures balance volatility using standard deviation and coefficient of variation to identify accounts with unstable balances that may indicate higher financial risk or irregular behavior.

```
balance_stats.to_csv("Task 4_2.csv")
```

4.3 Use IQR or z-score methods to detect anomalies

```
cleaned_df['Z_Score'] = (
    (cleaned_df['TransactionAmount'] -
cleaned_df['TransactionAmount'].mean()) /
    cleaned_df['TransactionAmount'].std()
)

anomalies = cleaned_df[abs(cleaned_df['Z_Score']) > 3]

anomaly_count = anomalies.groupby('AccountID').size().reset_index(
    name='AnomalyCount'
)

anomalies
#no anomalies found
```

```
Empty DataFrame
Columns: [TransactionID, CustomerID, AccountID, AccountType, TransactionType,
Product, Firm, Region, Manager, TransactionDate, TransactionAmount,
AccountBalance, RiskScore, CreditRating, TenureMonths, Month, Year,
Txn_Category, Gap, Inactive_Flag, CreditAmount, DebitAmount, TxnNature,
Z_Score]
Index: []

[0 rows x 24 columns]
```

This analysis uses the Z-score method to detect unusually large or abnormal transactions, helping identify potential anomalies or suspicious activities that may require further investigation.

4.4 Highlight customers with irregular or suspicious transaction behavior.

```
suspicious_df = large_withdrawal_count.merge(
    overdraft_count, on='AccountID', how='outer'
).merge(
    anomaly_count, on='AccountID', how='outer'
)
```


```
suspicious_df.fillna(0, inplace=True)

suspicious_df['SuspiciousFlag'] = suspicious_df.apply(
    lambda x: 'Suspicious' if (
        x['LargeWithdrawalCount'] > 0 or
        x['OverdraftCount'] > 0 or
        x['AnomalyCount'] > 0
    ) else 'Normal',
    axis=1
)

suspicious_customers = suspicious_df[
    suspicious_df['SuspiciousFlag'] == 'Suspicious'
]

suspicious_customers
```

```
   AccountID  LargeWithdrawalCount  OverdraftCount  AnomalyCount
0   ACC10996                   1.0             0.0           0.0
1   ACC11188                   1.0             0.0           0.0
2   ACC12334                   1.0             0.0           0.0
3   ACC15228                   2.0             0.0           0.0
4   ACC15359                   1.0             0.0           0.0
..       ...                   ...             ...           ...
90  ACC91723                   1.0             0.0           0.0
91  ACC92558                   1.0             0.0           0.0
92  ACC94242                   2.0             1.0           0.0
93  ACC94907                   1.0             0.0           0.0
94  ACC99409                   1.0             0.0           0.0

   SuspiciousFlag
0      Suspicious
1      Suspicious
2      Suspicious
3      Suspicious
4      Suspicious
..            ...
90     Suspicious
91     Suspicious
92     Suspicious
93     Suspicious
94     Suspicious

[95 rows x 5 columns]
```

This step combines multiple risk indicators—large withdrawals, overdrafts, and anomalies—to flag customers with suspicious transaction behavior for focused monitoring and further investigation.

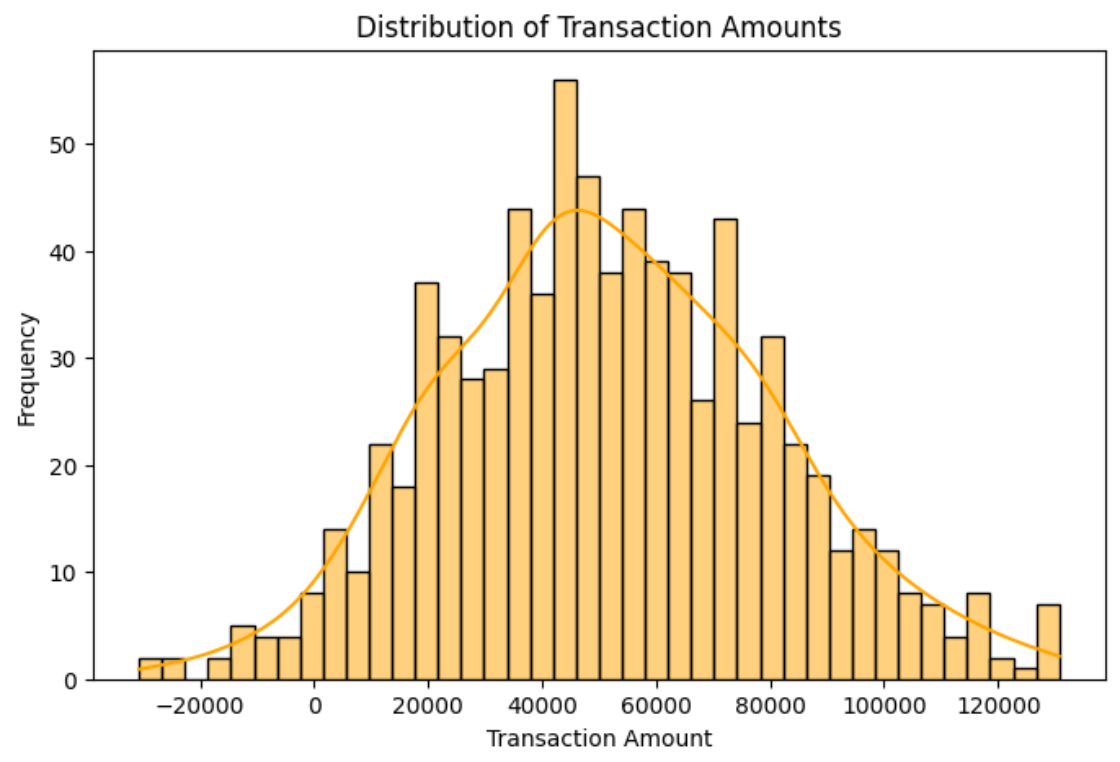
```
suspicious_customers.to_csv("Task 4_3.csv")
```

Task 5: Visualisation

Conduct extensive exploratory data analysis with attractive visualizations for your findings

Distribution of Transaction Amounts

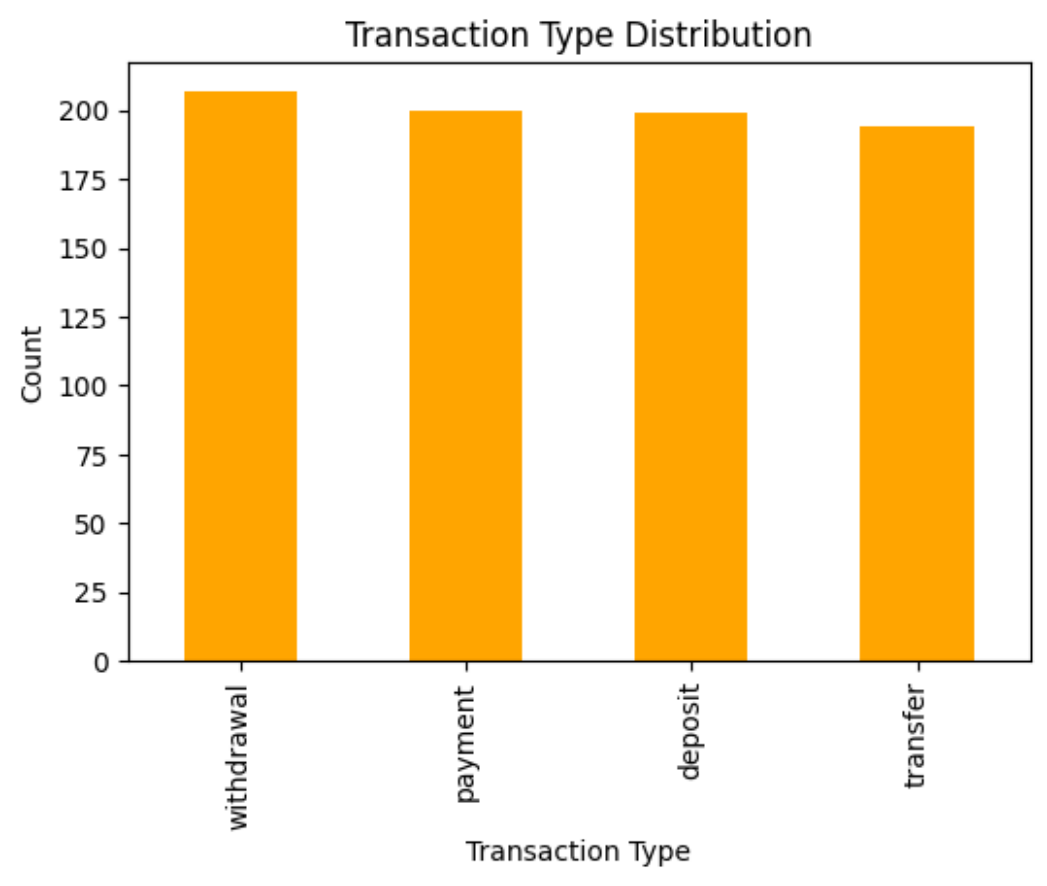
```
plt.figure(figsize=(8,5))
sns.histplot(cleaned_df['TransactionAmount'], bins=40, kde=True,
color="orange")
plt.title("Distribution of Transaction Amounts")
plt.xlabel("Transaction Amount")
plt.ylabel("Frequency")
plt.show()
```



This graph shows that most transactions are of medium value, while very small and very large transactions are less frequent, indicating a normal spending pattern with few extremes.

Transaction Type Distribution

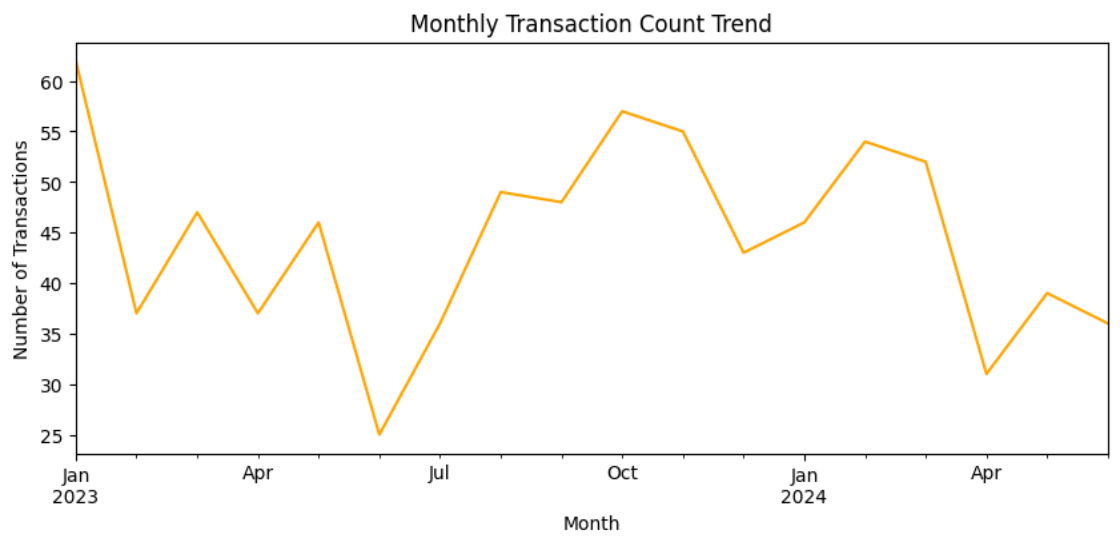
```
txn_type_counts = cleaned_df['TransactionType'].value_counts()

plt.figure(figsize=(6,4))
txn_type_counts.plot(kind='bar', color="orange" )
plt.title("Transaction Type Distribution")
plt.xlabel("Transaction Type")
plt.ylabel("Count")
plt.show()
```



This graph shows that all transaction types occur in similar numbers, indicating a balanced mix of withdrawals, payments, deposits, and transfers.

Monthly Transaction Count Trend

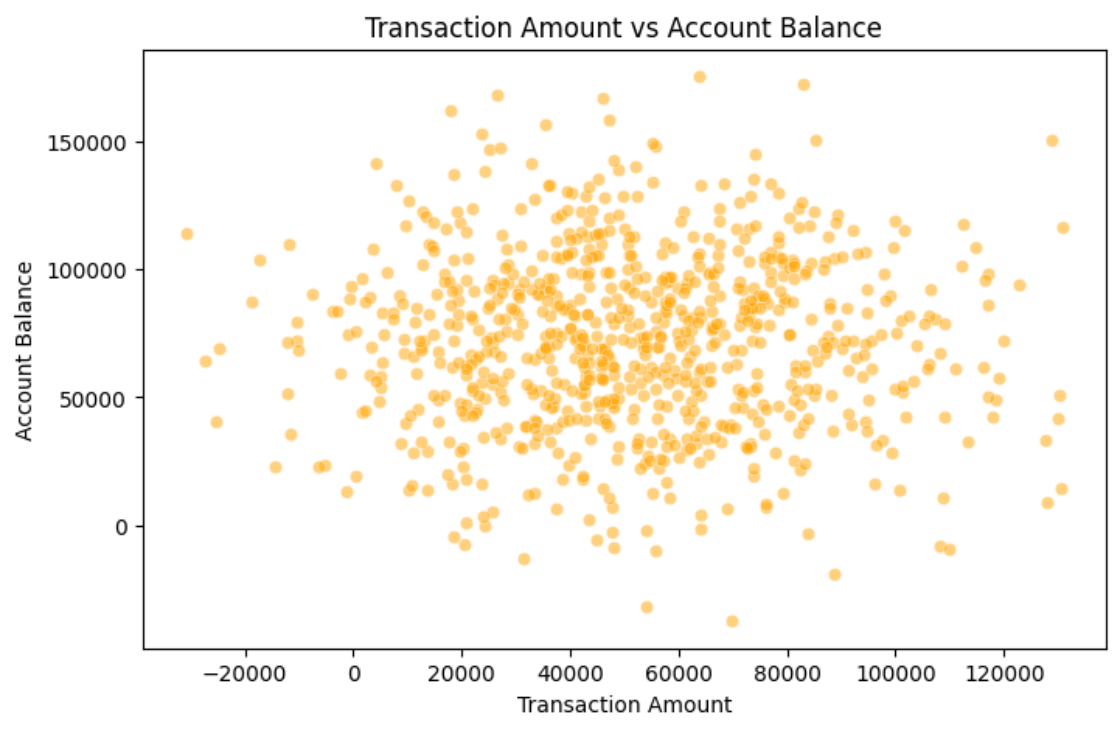
```
monthly_txn_count = cleaned_df.groupby('Month').size()

plt.figure(figsize=(10,4))
monthly_txn_count.plot(color = "orange")
plt.title("Monthly Transaction Count Trend")
plt.xlabel("Month")
plt.ylabel("Number of Transactions")
plt.show()
```



This graph shows that the number of transactions changes from month to month, indicating seasonal or periodic variations in customer activity.

Transaction Amount vs Account Balance

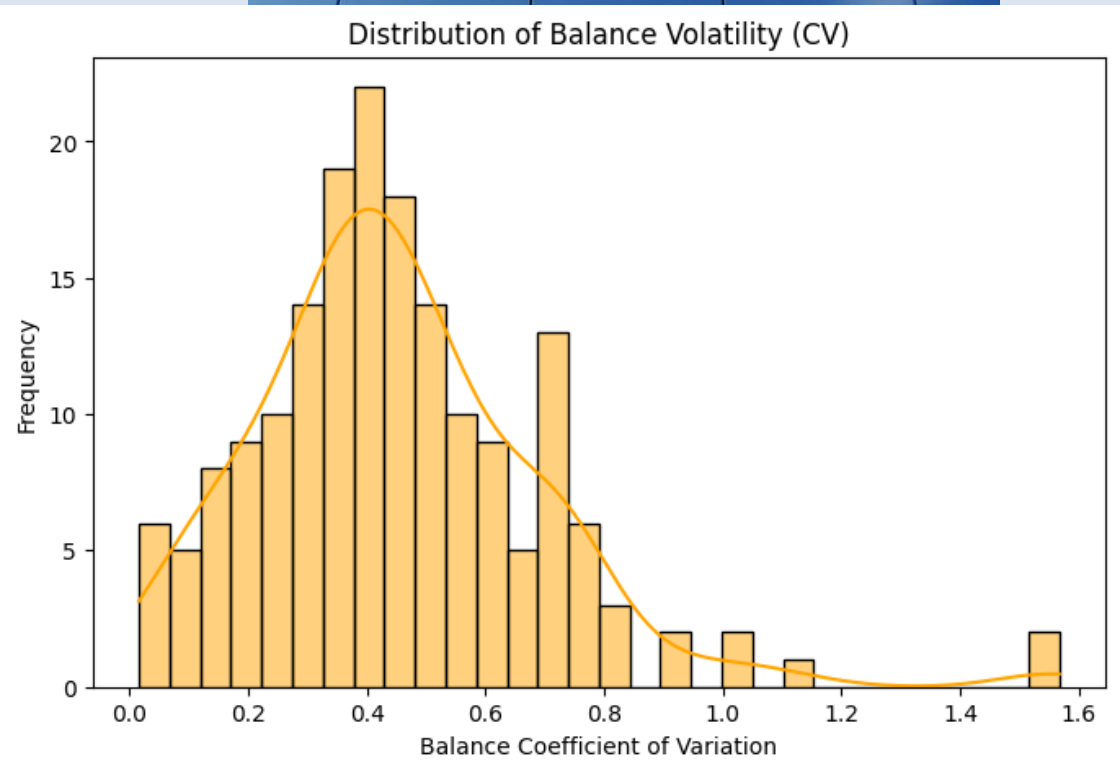
```
plt.figure(figsize=(8,5))
sns.scatterplot(
    x=cleaned_df['TransactionAmount'],
    y=cleaned_df['AccountBalance'],
    alpha=0.5, color = "orange"
)
plt.title("Transaction Amount vs Account Balance")
plt.xlabel("Transaction Amount")
plt.ylabel("Account Balance")
plt.show()
```



This graph shows no strong direct relationship between transaction amount and account balance, meaning high transactions do not always result in high balances.

Distribution of Balance Volatility (CV)

```
plt.figure(figsize=(8,5))
sns.histplot(balance_stats['CV'], bins=30, kde=True, color= "orange")
plt.title("Distribution of Balance Volatility (CV)")
plt.xlabel("Balance Coefficient of Variation")
plt.ylabel("Frequency")
plt.show()
```

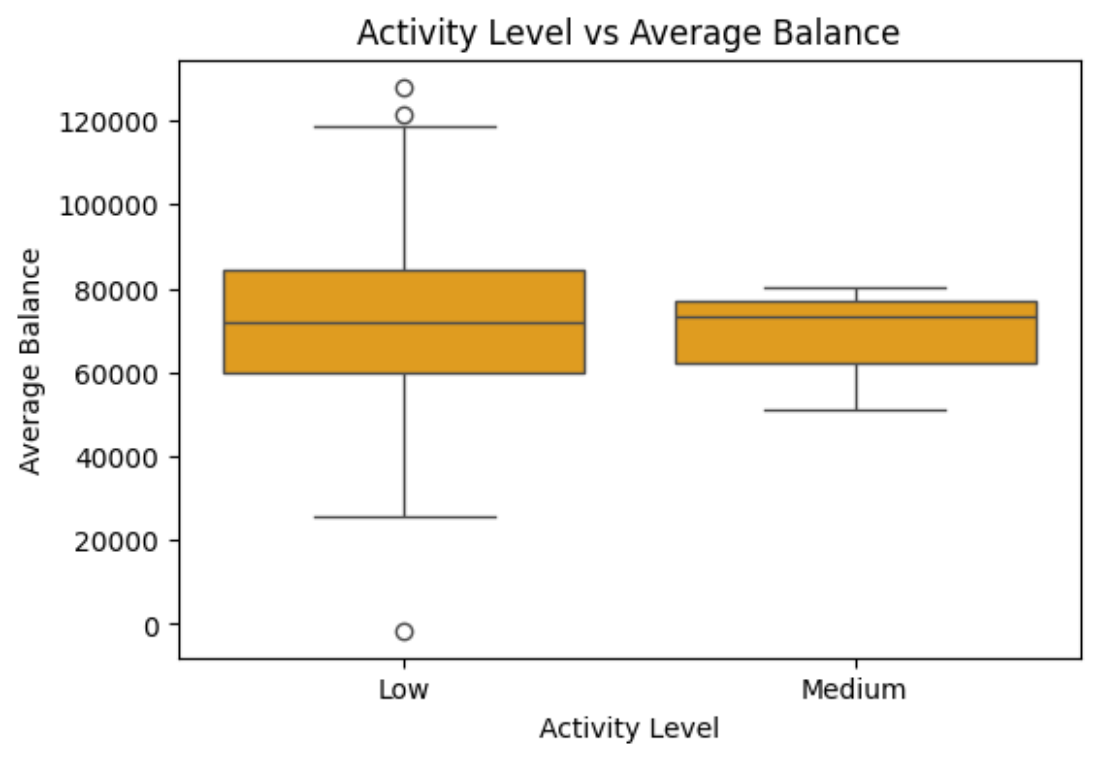


This graph shows that most accounts have moderate balance changes, while a few accounts have very high volatility, indicating unstable or risky balance behavior.

Activity Level vs Average Balance

```
transction_frequency = (
    cleaned_df
    .groupby('AccountID')
    .size()
    .reset_index(name='TxnCount')
)
def activity_level(count):
    if count > 20:
        return 'High'
    elif count >= 10:
        return 'Medium'
    else:
        return 'Low'

transction_frequency['ActivityLevel'] = (
    transction_frequency['TxnCount'].apply(activity_level)
)
activity_balance = transction_frequency.merge(
```

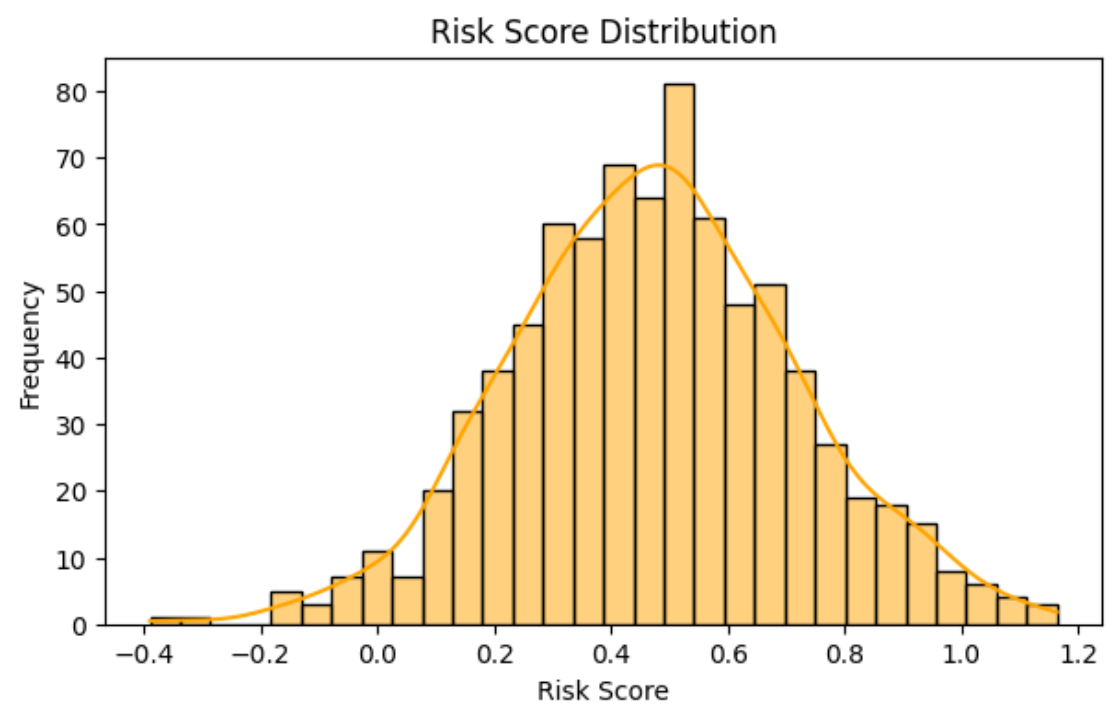

```
    cleaned_df.groupby('AccountID')['AccountBalance']
        .mean()
        .reset_index(),
    on='AccountID'
)
plt.figure(figsize=(6,4))
sns.boxplot(
    x=activity_balance['ActivityLevel'],
    y=activity_balance['AccountBalance'],
    color= "orange"
)
plt.title("Activity Level vs Average Balance")
plt.xlabel("Activity Level")
plt.ylabel("Average Balance")
plt.show()
```



This graph shows that medium-activity accounts generally maintain more stable and higher average balances, while low-activity accounts have wider balance variation and more extreme values.

Risk Score Distribution

```
plt.figure(figsize=(7,4))
sns.histplot(df['RiskScore'], bins=30, kde=True, color="orange" )
plt.title("Risk Score Distribution")
plt.xlabel("Risk Score")
plt.ylabel("Frequency")
plt.show()
```



This graph shows that most customers have a medium risk score, while only a small number fall into very low or very high risk categories.

task 6 Hypothesis Testing

6.1 and 6.2

Test whether high-volume transaction accounts have statistically higher average

balances than low-volume accounts.

Conduct hypothesis testing based on segmentation.

```
# account-level summary

account_summary = cleaned_df.groupby('AccountID').agg(
    Transaction_Count=('TransactionAmount', 'count'),
    Avg_Balance=('AccountBalance', 'mean')
).reset_index()

print("\nAccount-level summary")
print(account_summary.head())
```

```
Account-level summary
  AccountID  Transaction_Count   Avg_Balance
0  ACC10117                  4  70107.007957
1  ACC10996                  5  43568.008084
2  ACC11062                  2  38137.132610
3  ACC11188                  5  69652.151044
4  ACC11285                  3  97401.348560
```

```
# Segment accounts by transaction volume



avg_txn_count = account_summary['Transaction_Count'].mean()

high_volume_accounts = account_summary[
    account_summary['Transaction_Count'] > avg_txn_count
]

low_volume_accounts = account_summary[
```


```
    account_summary['Transaction_Count'] <= avg_txn_count
]

print("\nSegmentation results")
print("Average transaction count:", round(avg_txn_count, 2))
print("High-volume accounts:", high_volume_accounts.shape[0])
print("Low-volume accounts:", low_volume_accounts.shape[0])
```

```
Segmentation results
Average transaction count: 4.12
High-volume accounts: 73
Low-volume accounts: 121
```

```
# Compare average balances

high_avg_balance = high_volume_accounts['Avg_Balance']
low_avg_balance = low_volume_accounts['Avg_Balance']

print("\nAverage balances")
print("High-volume avg balance:", round(high_avg_balance.mean(), 2))
print("Low-volume avg balance:", round(low_avg_balance.mean(), 2))
```

```
Average balances
High-volume avg balance: 72512.19
Low-volume avg balance: 71683.28
```

```
# Perform TWO-SAMPLE T-TEST

t_stat, p_value = ttest_ind(
    high_avg_balance,
    low_avg_balance,
    equal_var=False
)

print("\nHypothesis test results")
print("T-statistic:", round(t_stat, 4))
print("P-value:", round(p_value, 4))
```

```
Hypothesis test results
```


```
T-statistic: 0.3151
P-value: 0.7531


# Hypothesis decision

alpha = 0.05

print("\nDecision")
if p_value < alpha:
    print("Reject H0: High-volume accounts have significantly higher average
balances.")
else:
    print("Fail to reject H0: No significant difference in average
balances.")



# DISPLAY TABLES
print("\nHigh-volume accounts sample")
display(high_volume_accounts.head())
print("\nLow-volume accounts sample")
display(low_volume_accounts.head())


Decision
Fail to reject H0: No significant difference in average balances.

High-volume accounts sample

  AccountID  Transaction_Count   Avg_Balance
1  ACC10996                  5  43568.008084
3  ACC11188                  5  69652.151044
7  ACC12334                  6  78082.517883
8  ACC13357                  6  69179.806513
9  ACC15228                  5  95389.614144


Low-volume accounts sample

  AccountID  Transaction_Count   Avg_Balance
0  ACC10117                  4  70107.007957
2  ACC11062                  2  38137.132610
4  ACC11285                  3  97401.348560
```

5	ACC11837	4	84852.733695
6	ACC12182	4	83837.145895

This hypothesis test compares high-volume and low-volume transaction accounts and finds no statistically significant difference in their average balances, as the p-value is greater than 0.05.

based on hypothesis testing, there is no statistically significant difference in average balances between high-volume and low-volume transaction accounts, indicating that transaction frequency alone does not strongly influence account balance levels.

Recommendations

The bank should closely monitor accounts with irregular behavior, such as frequent large withdrawals, overdrafts, high balance volatility, or anomalous transactions, to reduce financial risk.

High-value and high-activity customers can be targeted with personalized offers, premium services, or retention strategies.

Inactive or low-engagement accounts should be identified for re-engagement campaigns or operational review.

Implementing automated alerts and dashboards based on these risk indicators can improve fraud detection and decision-making efficiency.

VIDEO LINK